Person Re-Identification in Video

Complete Project Guide: Fundamentals, Implementation, Operation, and Evaluation

Project: Person re-identification in a single-camera video
Project folder: `Assignment/person_reid_video`
Implementation: Python, PyTorch, Ultralytics YOLO, TorchReID OSNet, FaceNet-PyTorch, OpenCV, SciPy
Guide version: 29 October 2026

> This is an educational prototype. A green box means the software accepted a similarity-based match under its current settings; it is not proof of a person's identity. Obtain permission to process identifiable imagery, and do not use this prototype for consequential decisions.

Contents

1. Project at a glance
2. Fundamentals: detection, tracking, and re-identification
3. System architecture
4. Models and feature representations
5. How the matching decision works
6. How temporal tracking and green highlighting work
7. Project files and responsibilities
8. Environment and installation
9. Preparing the reference gallery
10. Running one video
11. Running multiple videos and multiple identities
12. Understanding the output video and CSV
13. Progress, speed, and troubleshooting
14. Observed project results
15. Testing, validation, and advanced evaluation
16. Limitations, privacy, and responsible use
17. Glossary
18. Step-by-step quick checklist

---

1. Project at a glance

This project takes:

- a video containing people;
- a reference gallery containing one or more images per labeled person.

It detects person-shaped regions in each video frame, describes each region with a body-appearance feature vector, optionally checks a face, compares these features with the gallery, maintains short-term track numbers, and saves:

1. an annotated MP4 video;
2. a CSV table with one row per detected person in each frame.

The current implementation uses these default settings:

Component or setting	Current default
Person detector	Ultralytics YOLOv8-n, pretrained COCO weights
Detector image size	640 pixels
Person detection confidence	0.25
Body ReID embedder	OSNet AIN x1.0, pretrained on MSMT17
Face detector	MTCNN from FaceNet-PyTorch
Face embedding model	InceptionResnetV1, pretrained on VGGFace2
Body-only identity threshold	0.75 cosine similarity
Face-assisted threshold	0.70 cosine similarity
Minimum body similarity for face-assisted matching	0.35
Face checks per frame	Up to 5 high-scoring person detections
Repeat face check for a track	At least every 10 frames
Track association maximum age	30 missed frames
Track association minimum appearance similarity	0.55
Progress log interval	Every 5 frames
Device selection	`auto`: CUDA if available, Apple MPS if available, otherwise CPU

Thresholds are prototype defaults, not universally correct values. They have not been calibrated using an independently labeled validation set.

What the result means

- **Green box:** a face/body comparison accepted an identity for this detection, or this detection is associated with a track that was previously verified.
- **Blue/orange box:** a body-only match passed the higher body threshold. This state is supported by clothing/body appearance, not an accepted face comparison.
- **Yellow/orange box:** no match passed the configured rule; the identity is `Unknown`.
- **Track ID:** a temporary number for associating detections through nearby frames. It is not a real-world identity and is not the gallery label.

The color values are drawn in OpenCV's BGR format. On screen, confirmed detections appear green, body-only detections appear blue/orange, and unknown detections appear orange/yellow.

2. Fundamentals: detection, tracking, and re-identification

These are related but different computer-vision tasks.

2.1 Object/person detection

Detection answers: **"Where are the people in this frame?"**

The detector processes one frame at a time and returns bounding boxes, class labels, and detector confidence scores. The project keeps the COCO person class (`classes=[0]`). A box is a rectangle represented as:

```
(x1, y1, x2, y2)
```

Here, `(x1, y1)` is the top-left corner and `(x2, y2)` is the bottom-right corner in pixels. The confidence filter removes low-confidence detector predictions. This confidence is not the same as identity similarity.

2.2 Tracking

Tracking answers: **"Which detection in this frame is probably the same moving subject as a detection in a nearby frame?"**

The tracker assigns a temporary integer such as `ID 18` and attempts to preserve it between frames. It uses appearance features, overlap between boxes, and Hungarian assignment to avoid assigning one detection to multiple existing tracks.

Tracking is temporal and local to this one video. The tracker does not establish a person's actual identity and does not connect track IDs across separate videos.

2.3 Person re-identification (ReID)

Person ReID answers: **"Does this person's visual appearance resemble an entry in the labeled gallery?"**

The system extracts a numeric embedding from each detected person crop and compares it with gallery embeddings. ReID is a similarity task, not a database lookup of a verified name. It can fail when clothing, lighting, viewpoint, pose, scale, image quality, or occlusion differ.

2.4 Face matching

The optional face branch answers: **"Does a visible face crop resemble a face in the gallery?"**

MTCNN attempts to find and align a face inside a person crop. FaceNet then converts that aligned face into an embedding. Face matching can help when a face is visible, but it can fail when faces are small, blurry, turned away, occluded, or poorly lit. It should not be treated as certainty.

2.5 Open-set matching and `Unknown`

The person in a video may not appear in the gallery. An open-set system must be allowed to reject a candidate instead of forcing every detection into a known identity. This project labels a detection `Unknown` if it fails the available matching rules.

---

3. System architecture



Processing sequence for a frame

1. Read the next frame from the input video.
2. Ask YOLO to detect people.
3. Clip each detected box to the frame boundaries and discard invalid crops.
4. Convert each crop from OpenCV BGR to RGB for the embedding models.
5. Extract an OSNet body embedding for each crop.
6. Compare body embeddings with all gallery identity templates using cosine similarity.
7. From the highest body scores, select up to five candidates whose body similarity is at least `0.35`, subject to the 10-frame-per-track check interval.
8. Run MTCNN and FaceNet on selected crops where faces can be detected.
9. Compare each face embedding with gallery face templates.

10. Accept a face-assisted identity only when the best body identity and best face identity agree and both similarity thresholds are met.
11. Otherwise, accept a body-only identity only if its body similarity reaches `0.75`. If neither rule accepts a candidate, label it `Unknown`.
12. Associate detections with temporary tracks.
13. Persist a green identity on a track already verified by face/body agreement.
14. Draw the box and label; write the per-frame detection row into the CSV.
15. Write the annotated frame to the output video and periodically report progress.

---

4. Models and feature representations

4.1 YOLOv8-n person detector

YOLO ("You Only Look Once") is a one-stage object detector. It predicts object boxes and class confidence in a single forward pass. The `n` model is the nano-sized YOLOv8 variant: selected here to reduce compute requirements. The pretrained COCO model knows the generic `person` category; it is not trained to recognize the user's particular target.

The project loads detector weights named `yolov8n.pt`. If the weights are not present, Ultralytics may download them on the first run. The detector input size is `640` by default; increasing it can help detect smaller people, but increases memory use and inference time.

4.2 OSNet body ReID

OSNet (Omni-Scale Network) is designed to represent a person's appearance at multiple spatial scales. This project loads OSNet AIN x1.0 weights trained on MSMT17, a person ReID dataset.

The model produces a 512-element body feature vector per person crop. The vector is L2-normalized. A simplified way to think about the vector is a compact numerical signature of appearance. It is not a face template and does not encode a guaranteed identity.

The OSNet checkpoint is stored under the project's ignored `weights` folder. `embedder.py` verifies the downloaded checkpoint's SHA-256 checksum before use. The project converts BGR crops to RGB before handing them to the feature extractor.

4.3 MTCNN and FaceNet

MTCNN is a multi-stage convolutional network that detects a face, estimates landmarks, and aligns/crops the face. The configured crop size is 160 by 160 pixels.

FaceNet's InceptionResnetV1 model, pretrained on VGGFace2, maps a detected face to a 512-dimensional feature vector. The vector is L2-normalized before comparison. Face embeddings are calculated locally in the project process; they are not sent to an identity service.

If a gallery identity has no image where MTCNN can detect a face, that identity has no face template. Body-only matching remains possible.

4.4 Gallery templates

For each immediate subfolder in the gallery:

- all supported images are loaded;
- OSNet computes an embedding per image;
- the embeddings are averaged and normalized into one body template;
- FaceNet computes an embedding for each image where MTCNN finds a face;
- detected face embeddings are averaged and normalized into one face template.

Consequently, do not mix different people in the same folder. All images in one folder are treated as the same label. Different people must be in separate folders.

4.5 Cosine similarity

Cosine similarity compares the direction of two vectors:

```
cosine_similarity(a, b) = (a · b) / (||a|| × ||b||)
```

With unit-length normalized vectors, this reduces to their dot product. Larger values mean the vectors point in more similar directions. Scores are not probabilities: a score of `0.80` does not mean "80% certain."

The project reports separate body and face similarity values. The thresholds for the two models are different and should not be compared as if they were the same scale.

---

5. How the matching decision works

5.1 Body comparison

For each detected person, the program compares its OSNet feature with every gallery body template. It finds the gallery index with the largest body similarity.

5.2 Face comparison

Face checks are used to limit runtime:

- only the top five body-scoring detections in a frame are considered;
- candidates must have body similarity of at least `0.35`;
- a track is not checked again until 10 frames have passed since its previous check;
- MTCNN must successfully detect a face in the crop.

When a face is found, the face embedding is compared with each available gallery face template.

5.3 Decision rules

The rules are applied in this order:

1. **Face + body accepted:** the best body identity and best face identity are the same; face cosine similarity is at least `0.70`; body cosine similarity is at least `0.35`.
2. **Body-only accepted:** if face/body verification did not accept and the best body similarity is at least `0.75`, assign the best body identity.
3. **Unknown:** otherwise reject the detection.

The current implementation assumes that the same gallery ordering is used for body and face templates. The agreement test explicitly requires the best face identity index to equal the best body identity index.

5.4 Why not lower every threshold?

Lower thresholds accept more candidates, but can also incorrectly label a different person. A body crop can score highly because of similar clothing or a partial sleeve. A face crop can score poorly because of blur or a side angle. Thresholds should be chosen using labeled examples from the intended camera and conditions, not just one promising frame.

5.5 Verified track highlighting

When a detection passes the face+body rule, the track number and gallery identity are recorded as verified. Later detections are outlined green while the tracker continues assigning them to that same track. These labels use the text `confirmed track`.

This is a display continuity rule, not a new independent face comparison on every green frame. If the tracker changes or switches IDs, the green highlight can stop or—in an ID-switch failure—follow the wrong detection. Inspect the video and CSV. Numeric track IDs are not globally unique between runs.

---

6. How temporal tracking and green highlighting work

The tracker in `tracker.py` uses four main ideas:

1. **Track state:** active track ID, most recent box, smoothed appearance feature, and missed-frame count.
2. **Cosine similarity:** compares current OSNet detection features to each active track's appearance feature.
3. **IoU (Intersection over Union):** measures overlap between a track's old box and a current detection box.
4. **Hungarian assignment:** solves a minimum-cost one-to-one assignment problem between active tracks and current detections.

The tracker permits a match if appearance similarity reaches its minimum (`0.55`) or box IoU reaches its minimum (`0.10`). Its cost combines appearance distance and box-overlap distance; appearance has weight `0.70`, and box overlap has weight `0.30`. Once a match is made, the track's feature is smoothed with momentum `0.80`. Tracks can remain eligible for association for up to 30 missed frames.

If a detection cannot be assigned to an existing track, it receives the next available integer track ID. If an active track is missed too long, it expires. This is a lightweight short-gap, single-camera tracker—not a general multi-camera tracking system.

The gallery label follows a verified track's ID so the box remains green between face checks. If a tracker ID is wrong, that visual continuation can also be wrong. Treat it as a continuity aid in a demo, not identity proof.

---

7. Project files and responsibilities

File/folder	Responsibility
`reid_video.py`	Command-line options, device selection, gallery loading, video loop, YOLO inference, matching rules, tracking calls, annotation, CSV writing, progress logs
`embedder.py`	Verified OSNet model checkpoint, RGB conversion, normalized body embeddings
`face_embedder.py`	MTCNN detection/alignment and normalized FaceNet embeddings
`tracker.py`	Appearance + IoU assignment, temporary track IDs, feature smoothing, missed-frame expiry
`tests/test_tracker.py`	Tests for track assignment, gaps, expiry, invalid inputs
`tests/test_reid_video.py`	Tests for match rules, verified-track display persistence, duration formatting
`requirements.txt`	Python dependency list, including pinned PyTorch/TorchVision and FaceNet-PyTorch 2.5.3
`.gitignore`	Ignores virtual environment, local gallery, outputs, model weights, caches
`data/gallery/`	Local reference images (ignored by Git)
`outputs/`	Annotated videos, CSVs, previews, temporary generated files (ignored by Git)
`weights/`	Downloaded OSNet checkpoint (ignored by Git)
`README.md`	Concise setup and usage instructions
`report_draft.md`	Assignment report draft and measured observations

8. Environment and installation

8.1 Requirements

The checked development environment uses Python 3.12.6, PyTorch 2.14.1, and TorchVision 0.29.1. Core packages include OpenCV, NumPy, SciPy, Ultralytics, TorchReID, and FaceNet-PyTorch.

The FaceNet-PyTorch dependency is pinned to 2.5.3. Version 2.6.0 requests older PyTorch/TorchVision versions and conflicts with the project's pinned PyTorch 2.14.1/TorchVision 0.29.1.

8.2 Create the virtual environment

Open the project folder in VS Code. Select **Terminal** → **New Terminal** and check that the terminal's current directory is `Assignment\person_reid_video`. Then run:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

If `.venv` already exists and the installation has completed, do not recreate it unnecessarily. A direct invocation that does not require activation is:

```
.\.venv\Scripts\python.exe --version
```

8.3 CPU and GPU

The checked project environment has CPU-only PyTorch. The observed machine has an NVIDIA GeForce RTX 3050 Laptop GPU, but the prior CUDA wheel installation did not complete successfully. With `--device auto`, the program selects CUDA only if the installed PyTorch build supports it; otherwise it runs on CPU.

The first run may download detector/model checkpoints. A slow internet connection can delay this initial setup. Do not copy model files from untrusted sources. OSNet download validation checks its SHA-256; Ultralytics and FaceNet have their own model-weight download behavior.

---

9. Preparing the reference gallery

9.1 Folder structure

Each immediate subfolder under `data\gallery` defines one identity label:

```
person_reid_video\
  data\
    gallery\
      person_01\
        front.jpg
        side.png
      person_02\
        reference.jpeg
```

Here, `person_01` and `person_02` are labels, not names recognized by the model. The program treats every supported image in `person_01` as a reference for the same person. `person_02` is a different identity.

9.2 Supported image formats

The code accepts `.jpg`, `.jpeg`, `.png`, `.bmp`, and `.webp` files. It scans images in each identity's immediate subfolder. It does not recursively search nested subfolders.

9.3 Reference-image quality

For best results:

- use clear, well-lit photos;

- include the full person for useful body-ReID information;
- include a visible, reasonably frontal face for face matching;
- use multiple views of the same person when authorized;
- do not put two different people in one identity folder;
- avoid group images and heavily edited images;
- do not use sensitive images without permission.

More gallery images can provide coverage of different views, but do not guarantee better performance. Averages can be less useful if images vary widely or the folder includes the wrong person.

9.4 Replacing a reference person

If a folder contains the previous person's images plus the new person's image, the program will treat all of them as one identity. To search only for a new person, move the old image out of that identity folder or create a fresh folder such as `person_02`. Keep each person's images together under their own identity folder.

---

10. Running one video

10.1 Place the video

The input video can remain in another folder. You only need to provide its full path. Supported container/codec combinations depend on the OpenCV build and installed video codecs; MP4/H.264 is a practical choice.

10.2 Open the terminal at the project folder

In VS Code, open the `person_reid_video` folder, then select **Terminal** → **New Terminal**. Use this check:

```
Get-Location
```

The path should end with:

```
Assignment\person_reid_video
```

10.3 Run the program

Replace the example video path with the actual location of your video:

```
.\.venv\Scripts\python.exe .\reid_video.py `
  --video "D:\Videos\my-video.mp4" `
  --gallery .\data\gallery `
  --output .\outputs\my-video-result.mp4 `
  --csv .\outputs\my-video-results.csv `
  --device auto
```

The backtick at the end of each continued line is PowerShell syntax. Keep each backtick as the very last character of the line. Alternatively, run the command on one line:

```
.\.venv\Scripts\python.exe .\reid_video.py --video "D:\Videos\my-video.mp4" --gallery .\data\gallery --output .\outputs\my-video-result.mp4 --csv .\outputs\my-video-results.csv --device auto
```

10.4 View the output

When processing completes:

- open `outputs\my-video-result.mp4` in a media player to view boxes and labels;
- open `outputs\my-video-results.csv` in Excel or another spreadsheet program to inspect frame-by-frame data.

To open the output folder from the project terminal:

```
explorer.exe .\outputs
```

Do not use the output file while it is still being written. Wait for the terminal's final `Annotated video` and `Detection results` messages.

---

11. Running multiple videos and multiple identities

11.1 Multiple identities in one gallery

Use one folder per person under the same gallery root. One command compares the input video against all gallery subfolders:

```
data\gallery\
  person_01\front.jpg
  person_01\side.jpg
  person_02\front.jpg
```

Use `--gallery .\data\gallery`. The person-folder name becomes the output identity label.

11.2 Multiple video files in a folder

The program accepts one `--video` file per invocation. To run every MP4 in `Assignment\slides` sequentially, use PowerShell from the project folder:

```
Get-ChildItem -LiteralPath "..\slides" -Filter "*.mp4" -File |
  ForEach-Object {
    $name = $_.BaseName
```

```
    .\.venv\Scripts\python.exe .\reid_video.py `
      --video $_.FullName `
      --gallery .\data\gallery `
      --output ".\outputs\$name-result.mp4" `
      --csv ".\outputs\$name-results.csv" `
      --device auto

    if ($LASTEXITCODE -ne 0) {
      throw "Processing failed for $($_.Name)"
    }
  }
```

This scans MP4s directly inside `slides`, not nested folders. Output names are based on each video filename. Runs happen sequentially: the next video starts after the previous invocation exits. The gallery is reloaded for every run.

For videos in another directory, change `"..\slides"` to its path. To include other extensions, change the filter logic, for example:

```
Get-ChildItem -LiteralPath "..\slides" -File |
  Where-Object { $_.Extension -in @(".mp4", ".avi", ".mov", ".mkv") }
```

Check that output filenames are unique. Reusing an output path overwrites that output file.

---

12. Understanding the output video and CSV

12.1 Annotated MP4

Each detected person gets a box and a label with:

- the temporary tracker number (`ID 18`);
- the gallery label (`person_01`, `person_02`, or `Unknown`);
- body cosine similarity;
- face cosine similarity when that detection was checked and a face was found;
- the note `confirmed track` when the identity label is carried on the same verified track.

Colors:

Display	Meaning
Green	Face+body match accepted, or a detection on a previously verified track
Blue/orange	Body-only match passed the higher body threshold
Yellow/orange	`Unknown`; neither matching rule accepted the identity

If two boxes overlap, labels may overlap visually. A detection can be missing entirely if the person detector did not find a person box.

12.2 CSV schema

The CSV columns are:

Column	Meaning
`frame`	Zero-based frame index within this input video
`time_seconds`	Approximate video time, calculated as frame index divided by reported input FPS
`detection`	Detection index within that frame; zero-based
`track_id`	Temporary tracker ID within this run
`identity`	Gallery folder name if accepted, otherwise `Unknown`
`body_similarity`	Best OSNet body cosine similarity for this detection
`face_similarity`	Best FaceNet score when a face was found for this detection; blank otherwise
`match_method`	`face+body`, `track-confirmed`, `body`, or `unknown`
`x1`, `y1`, `x2`, `y2`	Bounding-box coordinates in pixels

CSV similarities are floating point scores, not calibrated probabilities. A blank face score means no face score was available for that detection; it does not mean zero similarity.

---

13. Progress, speed, and troubleshooting

13.1 Progress messages

The program logs progress every five frames by default. When video frame count is available, a log line includes:

- frames completed and total frames;
- percentage complete;
- average processed frames per second;
- estimated remaining time (ETA).

Example format:

```
INFO: Progress: 25/569 frames (4.4%), 0.42 FPS, ETA 00:21:35
```

The exact speed and estimate will change as the run proceeds. An ETA is based on average speed so far, not a guarantee.

To log more often:

```
--progress-interval 1
```

To log less often, use a larger integer. An already-running process does not load source changes made later. Restart the command to use new progress options or code.

13.2 Why processing is slow

The video pipeline runs person detection and body embeddings on every frame, and selected detections also go through MTCNN and FaceNet. Crowd scenes with many person boxes take longer because each frame requires more crop processing.

In the checked CPU environment, the 119-frame, high-density excerpt took about 283 seconds (approximately 0.42 FPS). This was a particularly crowded example and should not be assumed to predict all videos. On CPU, even a short clip may take several minutes; several long videos can take much longer.

13.3 The terminal appears stuck

1. Look for repeated `INFO: Progress:` lines. If the frame count advances, it is working.
2. Wait for the final `Annotated video` and `Detection results` messages.
3. If progress does not advance for a long time, check CPU/GPU activity and whether the terminal process still exists.
4. Press `Ctrl+C` only if you intend to stop. A stopped output video is partial and should be rerun before use.
5. If the process was started before progress logging was added, it will not show the new progress lines; restart it to use the current code.

13.4 Common errors

Symptom	Likely cause and response
`Input video does not exist`	Check the spelling and quoting of the `--video` path. Use the complete path if unsure.
`Gallery directory does not exist`	Run from the project folder or provide the correct `--gallery` path.
No gallery images found	Put supported images in a person subfolder, for example `data\gallery\person_01\reference.jpg`.
Image could not be read	Check file integrity and that the extension/content is a supported image format.
No face in gallery warning	The image can still contribute to body matching, but that identity has no face template. Use a clearer frontal face image if authorized.
CUDA requested but unavailable	Use `--device auto` or `--device cpu`; CUDA requires a CUDA-enabled PyTorch installation and compatible driver.
Output video cannot be created	Close any player holding the output file, check write permissions, and ensure the output folder exists.
Labels are all `Unknown`	The subject may be occluded, not in the gallery, or scores may be below thresholds. Inspect the CSV and use representative authorized gallery images; do not lower

Symptom	Likely cause and response
	thresholds without validation.
Incorrect person becomes green	A face/track association or model false match may have occurred. Review the video; verify the person and tune with held-out labeled examples.
Partial output after interruption	Restart the run using an output path that is not open in another application.

14. Observed project results

14.1 Earlier full-video baseline

The initial ResNet baseline was run end-to-end on the supplied 18.97-second clip:

Measure	Observed value
Frames	569
Person detections	5,414
Temporary track IDs	47
CPU speed	About 1.42 FPS
ResNet gallery threshold	0.75
Highest baseline similarity	0.6904
Accepted gallery detections	0

This result belongs to an earlier ImageNet-pretrained ResNet prototype. It is not a measurement of the current OSNet/FaceNet pipeline.

14.2 Current OSNet/FaceNet excerpt

The corrected system was run on a 119-frame excerpt (source time approximately 15–18.97 seconds):

Measure	Observed value
Person detections	1,509
Fresh face/body verifications	2
Verified green detections on one track	19 consecutive detections
Other detections left `Unknown`	1,490
Fresh match scores (body / face)	0.393 / 0.805 and 0.358 / 0.760
Elapsed time	282.86 seconds
CPU throughput	About 0.42 FPS

The annotated output and preview are saved as `outputs\target_match_excerpt.mp4` and `outputs\target_match_excerpt_preview.png`. The two fresh matches are visually consistent with the yellow-jacketed subject used for the reference image. This is a small qualitative demonstration; there are no ground-truth identity labels for the entire clip. Therefore it is not an accuracy result and does not establish a calibrated false-match rate.

The observation that a partial yellow sleeve can have a higher OSNet similarity than a full-person crop (0.617 versus 0.497 in one higher-resolution probe) illustrates why a body score alone should not be treated as proof.

---

15. Testing, validation, and advanced evaluation

15.1 Run the included tests

From the project terminal:

```
.\.venv\Scripts\python.exe -m pytest .\tests -q
```

The tests cover synthetic tracker inputs, appearance matching decisions, verified-track persistence, and time formatting. The latest recorded test result is 14 passed. These tests do not prove that the models are accurate on a new video.

Additional environment checks:

```
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe -m compileall -q .\reid_video.py .\embedder.py
.\face_embedder.py .\tracker.py .\tests
```

15.2 Build a small labeled validation set

For a meaningful threshold study:

1. Select an authorized video where you know which people are present.
2. Record ground-truth identities and, if possible, manually check person boxes in a sample of frames.
3. Keep gallery photos separate from validation photos/frames.
4. Include both gallery people and people who are absent from the gallery.
5. Include challenging conditions such as distance, lighting, pose, and partial occlusion.
6. Run the system at candidate threshold values and record false accepts and missed matches.
7. Choose thresholds using the validation set, then lock them before evaluating a separate held-out test set.
8. Report how the set was created, counts, splits, threshold, and the evaluation metrics.

Do not choose and report a threshold based on the test set. That would make the final test result optimistic.

15.3 Useful ReID metrics

- **Rank-1 accuracy:** proportion of query samples whose top-ranked gallery identity is correct.
- **mAP (mean average precision):** summarizes precision over ranked retrieval results across queries.
- **False-match rate:** proportion of non-gallery/unknown people incorrectly accepted as a known identity at a selected threshold.
- **Miss rate:** proportion of true gallery people that fail to be accepted.
- **Precision of accepted matches:** proportion of accepted identity labels that are correct.
- **Recall of accepted matches:** proportion of true target appearances detected and accepted.

For detection, use precision/recall or mAP only when annotated box ground truth exists. For tracking, use IDF1, ID switches, or fragmentation only when ground-truth track annotations are available. Do not infer these metrics from a pleasing-looking clip.

15.4 Threshold calibration

Collect scores for verified same-person pairs and different-person pairs from the validation data. Plot score distributions or evaluate precision/recall and false-match rate over thresholds. Pick a threshold based on the project's intended trade-off: false matches can be more problematic than missed matches. Evaluate face and body thresholds separately because their score distributions differ. Keep the final test set untouched until thresholds are fixed.

15.5 Performance improvements to investigate

Possible measured optimizations include:

- installing a compatible CUDA-enabled PyTorch build when supported by the machine;
- processing a selected subset of frames, while accounting for missed short appearances;
- lowering detector image size, accepting that small-person recall may decrease;
- limiting face checks further, accepting that some targets may wait longer for confirmation;
- using a validated video decoder/inference pipeline and batching crop embeddings where supported;
- measuring GPU memory, CPU utilization, inference time per model, and total wall time separately.

Do not claim a speed improvement without timing the same inputs and settings before and after the change.

---

16. Limitations, privacy, and responsible use

1. **Similarity is not identity proof.** A model score is only a comparison of learned appearance features.
2. **Appearance changes matter.** Clothing, pose, distance, camera, compression, lighting, and occlusion can change embeddings.
3. **Faces may be unavailable.** Small or blurred faces may not be detected, aligned, or represented reliably.
4. **Track IDs may switch.** Crowd overlap and missed detections can cause one ID to refer to another person.
5. **Thresholds are not calibrated.** Current defaults were explored on a small sample and need validation on representative data.
6. **Single-camera scope.** The tracker does not connect identities reliably across separate videos or cameras.
7. **No built-in ground truth.** The supplied clip has no annotations from which to compute benchmark accuracy.

8. **Compute cost.** The CPU pipeline is slow, especially in dense scenes.
9. **Consent and data security.** Process only images/video you are authorized to use. Keep gallery images and private video local, restrict access, and delete them when no longer needed.
10. **Not for consequential use.** Do not use this coursework demo to make decisions affecting a person's rights, access, safety, employment, or reputation.

---

17. Glossary

Term	Plain-language meaning
Appearance embedding	A numeric vector representing visual characteristics learned by a model
Bounding box	Rectangle around a detected object
BGR / RGB	Two channel orders for color images; OpenCV commonly uses BGR, while model inputs usually use RGB
Checkpoint / weights	Saved model parameters learned during training
Cosine similarity	Direction-based vector similarity score
Detection confidence	Detector's confidence that a box contains the requested class; not identity confidence
Embedding	Compact numeric feature representation
Face alignment	Cropping/rotating a face into a consistent orientation using detected landmarks
Gallery	Labeled reference image collection
Hungarian algorithm	Method to find a low-cost one-to-one assignment between tracks and detections
IoU	Intersection-over-Union; overlap divided by combined area
L2 normalization	Scaling a vector so its Euclidean length is one
mAP	Mean Average Precision, a ranking/detection evaluation metric depending on task
MSMT17	A person ReID dataset used to train the OSNet checkpoint
MTCNN	Multi-stage neural face detector and aligner
Open set	Setting where an input may belong to none of the known gallery identities
OSNet	Omni-Scale Network, a person ReID embedding model
ReID	Re-identification: matching a detected person's appearance against a gallery
Track	A short temporal sequence of detections believed to depict the same moving subject
Track ID	Temporary integer assigned by the video tracker
VGGFace2	Large face dataset used for FaceNet model pretraining

Term	Plain-language meaning
YOLO	Real-time object-detection model family

18. Step-by-step quick checklist

Before running

- [ ] Open the `person_reid_video` project folder in VS Code.
- [ ] Confirm the environment is installed; `.venv\Scripts\python.exe` exists.
- [ ] Place each person's authorized reference images in that person's own folder under `data\gallery`.
- [ ] Confirm all images in one person folder actually depict that same person.
- [ ] Know the full path of the input video.
- [ ] Choose output names that will not overwrite useful prior results.

Run

- [ ] Open **Terminal** → **New Terminal** at the project folder.
- [ ] Run `Get-Location` to verify the terminal folder.
- [ ] Run the command from Section 10, replacing the video path and output filenames.
- [ ] Watch for `Progress:` lines; wait for final output paths.
- [ ] Do not interrupt unless necessary. `Ctrl+C` produces incomplete output.

Review

- [ ] Open the output MP4 from the `outputs` folder.
- [ ] Check that green boxes actually enclose the intended target.
- [ ] Check yellow/orange boxes and `Unknown` results; a missed green match may be a false negative, not proof the person was absent.
- [ ] Open the CSV and review timestamps, similarity scores, and match methods.
- [ ] Do not report accuracy unless you have labeled ground truth and a held-out evaluation.

Reference outputs and further details

- Quick setup and command examples: `README.md`
- Assignment-style results and methodology: `report_draft.md`
- Matching implementation and all CLI options: `reid_video.py`
- Body feature implementation: `embedder.py`
- Face feature implementation: `face_embedder.py`
- Tracker behavior: `tracker.py`